C for Pebble Programmers - Complete Emery Syllabus

Platform: Pebble Time 2 (emery, rectangular 200 x 228) ? Language: C ? SDK: 4.33.1 ? Core skills: 7 ? Required activities: 40 ? Estimated focused time: 48h 35m

Purpose

Develop independent capability in UI Design, Idea Generation, Algorithmic Thinking, C Programming, Resource Literacy, Debugging and Testing, and Development Tools and Workflow. Progress through short, self-directed, example-based activities with explicit Taskwarrior prerequisites. The C programming track has eight activities (4A-4H), drawing on *Learning C with Pebble- and current Emery SDK conventions.

Evidence-based completion

Each task is complete when (a) its stated functionality or written product meets the learning objective, (b) a tangible artifact supports that claim-code, emulator output, API note, pseudocode, sketch, or debugging record-and (c) the student can independently explain the work. Keep a versioned source diff where coding is involved. Attempt normal and boundary cases. Time spent alone is not evidence of mastery.

Independent-work policy: No AI-generated code, answers, diagrams, or debugging solutions during activities. Consult the in-repository starter code, SDK documentation, compiler diagnostics, logs and Git. Future grading arrangements are separate from this syllabus.

C programming progression

Activity | Core C subject | Local reading | Local starting code
4A | Variables, Data Types, and Operators | Markdown reading | Complete Emery lesson
4B | Conditional Execution and Decision Making | Markdown reading | Complete Emery lesson
4C | Loops and Iteration | Markdown reading | Complete Emery lesson
4D | Functions and Program Structure | Markdown reading | Complete Emery lesson
4E | Arrays, Strings, and Bounds | Markdown reading | Complete Emery lesson
4F | Pointers, Indirection, and Memory Lifetime | Markdown reading | Complete Emery lesson
4G | Structures, Enums, and Program State | Markdown reading | Complete Emery lesson
4H | Event-Driven Integration and Application Lifecycle | Markdown reading | Complete Emery lesson

Book-based exercises from Chapters 3-6 now have 11 complete, in-repository project starters under lessons/4A/projects through lessons/4D/projects. Activities 4E-4H each supply their own independent C watchapp under lessons/<ID>/starter. The original historical chapter text is preserved in the first four readings with necessary Emery updates and links to these local examples.

Recommended phase order and exact dependencies

Phase I - Orientation and first C expressions

Activity | Estimate | Difficulty | Priority | Depends on
2A Explore Pebble examples | 20m | Low | High | None
7A Build and run a time-only C watchface | 30m | Low | High | None
4A Variables, Data Types, and Operators | 120m | Low | High | 7A
5A Find and document a Pebble API function | 20m | Low | High | 4A
1A Change one visual property of a time-only watchface | 30m | Low | High | 4A, 5A, 7A
6A Diagnose a simple compiler error | 20m | Low | High | 4A, 7A

7B Create a Git-based modification workflow | 45m | Medium | High | 7A, 1A, 6A

Phase II - Control flow, functions and interaction

Activity | Estimate | Difficulty | Priority | Depends on
4B Conditional Execution and Decision Making | 120m | Medium | High | 4A
4C Loops and Iteration | 135m | Medium | High | 4B
4D Functions and Program Structure | 150m | Medium | High | 4C
3A Trace the watchface lifecycle | 30m | Low | High | 4D, 7A
1B Add and style a date TextLayer | 45m | Medium | Medium | 1A, 4D, 5A
5B Trace an existing feature through source code | 45m | Medium | Medium | 3A, 4D, 5A
3B Design a font-cycling algorithm | 30m | Medium | Medium | 3A, 5B
2B Write a two-example remix brief | 30m | Medium | Medium | 2A, 5B

Phase III - Data, memory, state and original design

Activity | Estimate | Difficulty | Priority | Depends on
4E Arrays, Strings, and Bounds | 90m | Medium | High | 4D
4F Pointers, Indirection, and Memory Lifetime | 105m | High | High | 4E
4G Structures, Enums, and Program State | 105m | High | Medium | 4F
5C Research an unfamiliar Pebble capability | 60m | High | Medium | 4E, 5B
6B Diagnose a font-selection boundary bug | 45m | Medium | Medium | 4E, 6A, 3B
2C Develop an original Pebble app concept | 60m | High | Medium | 2B, 5C
3C Design a state machine for an app | 60m | High | Medium | 2C, 3B, 4G
4H Event-Driven Integration and Application Lifecycle | 150m | High | Medium | 4G, 3C
1C Design a multi-mode Pebble interface | 90m | High | Medium | 1B, 2C, 3C, 4H

Phase IV - Integration and independent verification

Activity | Estimate | Difficulty | Priority | Depends on
7C Integrate font browsing with persistent selection | 150m | High | Medium | 1C, 4H, 5B, 7B
6C Diagnose a persistent-state integration bug | 90m | High | Low | 4H, 6B, 7C

Detailed activities

Phase I - Orientation and first C expressions

2A - Explore Pebble examples

Skill: Idea Generation ? Time: 20m ? Difficulty: Low ? Priority: High ? Prerequisites: None

Exact starter and directions: lessons/2A/.

Do: Inspect the three included Emery-only projects under lessons/2A. Brainstorm five modifications and note what SDK feature each might require. No coding or debugging-project spoilers.

Complete when: Five distinct ideas are recorded, each describes a possible change, and one idea is marked as the most interesting to investigate.

7A - Build and run a time-only C watchface

Skill: Development Tools and Workflow ? Time: 30m ? Difficulty: Low ? Priority: High ? Prerequisites: None

Exact starter and directions: lessons/7A/.

Do: Create a new C project with the CURRENT watchface tutorial for SDK 4. Set package.json targetPlatforms to ["emery"]. Follow the tutorial through Displaying the Time and STOP before

Adding the Date. Run pebble build then pebble install --emulator emery; confirm time updates and only one TextLayer exists. Identify project structure and build output.

Complete when: The project builds and runs in the emulator, displays the current time, updates by minute, and has no date TextLayer.

4A - Variables, Data Types, and Operators

Skill: C Programming ? Time: 120m ? Difficulty: Low ? Priority: High ? Prerequisites: 7A

Exact starter and directions: lessons/4A/. Reading: 4A module.

Do: Read variables, types, operators and casts. Complete the bouncing ball and orbiting-letter projects, modifying expressions and interpreting their behavior.

Complete when: Demonstrate both Emery apps, explain types and arithmetic, and supply code diffs plus boundary tests.

5A - Find and document a Pebble API function

Skill: Resource Literacy ? Time: 20m ? Difficulty: Low ? Priority: High ? Prerequisites: 4A

Exact starter and directions: lessons/5A/.

Do: Find the SDK documentation for text_layer_set_font() or another function used by your watchface. Record what it does, its full function signature, the type and purpose of each argument, its return type, and one example of where it is used. Write the explanation in your own words.

Complete when: A concise API note contains purpose, signature, argument meanings, return type, and a usage example.

1A - Change one visual property of a time-only watchface

Skill: UI Design ? Time: 30m ? Difficulty: Low ? Priority: High ? Prerequisites: 4A, 5A, 7A

Exact starter and directions: lessons/1A/.

Do: Use lessons/1A/starter with only one time TextLayer. Change exactly one of font, color, alignment or position; verify that the clock still updates.

Complete when: The app builds and runs, the time still updates, and one intentional visual change is visible. The watchface still has no date layer.

6A - Diagnose a simple compiler error

Skill: Debugging and Testing ? Time: 20m ? Difficulty: Low ? Priority: High ? Prerequisites: 4A, 7A

Exact starter and directions: lessons/6A/.

Do: Use the deliberately flawed local lessons/6A/starter project. Capture the first compiler diagnostic, make the minimal syntax correction, then build and run on Emery.

Complete when: You can show the original error, identify its cause, fix it, and produce a successful build.

7B - Create a Git-based modification workflow

Skill: Development Tools and Workflow ? Time: 45m ? Difficulty: Medium ? Priority: High ? Prerequisites: 7A, 1A, 6A

Exact starter and directions: lessons/7B/.

Do: Make a personal Git branch or working copy of lessons/7B/starter, commit a baseline, change one visual property, build/test and commit the improvement.

Complete when: The repository has a baseline commit and a modification commit, the app builds, and you have inspected the diff or demonstrated how to restore the baseline.

Phase II - Control flow, functions and interaction

4B - Conditional Execution and Decision Making

Skill: C Programming ? Time: 120m ? Difficulty: Medium ? Priority: High ? Prerequisites: 4A

Exact starter and directions: lessons/4B/. Reading: 4B module.

Do: Read if/else, switch, logical operators and short-circuiting. Complete the sweeping-seconds and digital-clock decision exercises.

Complete when: Two modified Emery builds plus an explanation of each branch and tests at decision boundaries.

4C - Loops and Iteration

Skill: C Programming ? Time: 135m ? Difficulty: Medium ? Priority: High ? Prerequisites: 4B

Exact starter and directions: lessons/4C/. Reading: 4C module.

Do: Study for, while, do/while and nested loops. Complete decimal-to-binary, color-grid replacement and screen-filling text exercises.

Complete when: All three exercises run on Emery; the loop bounds and termination conditions are explained and verified.

4D - Functions and Program Structure

Skill: C Programming ? Time: 150m ? Difficulty: Medium ? Priority: High ? Prerequisites: 4C

Exact starter and directions: lessons/4D/. Reading: 4D module.

Do: Study signatures, parameters, return values, callbacks and recursion. Complete the clock-helper, bit-pattern, rectangle and concentric-circle projects.

Complete when: Four project artifacts show correct function contracts, a terminating recursive function and a working build.

3A - Trace the watchface lifecycle

Skill: Algorithmic Thinking ? Time: 30m ? Difficulty: Low ? Priority: High ? Prerequisites: 4D, 7A

Exact starter and directions: lessons/3A/.

Do: Draw the execution flow for the time-only watchface. Include main(), init(), window_stack_push(), main_window_load(), update_time(), tick_handler(), app_event_loop(), main_window_unload(), and deinit() where applicable. Show which functions call other functions and which are callbacks. Show the separate path for a minute tick after startup.

Complete when: A flow diagram shows startup, initial display, minute updates, and cleanup, with direct calls distinguished from callbacks.

1B - Add and style a date TextLayer

Skill: UI Design ? Time: 45m ? Difficulty: Medium ? Priority: Medium ? Prerequisites: 1A, 4D, 5A

Exact starter and directions: lessons/1B/.

Do: Start from the time-only watchface in lessons/1B/starter and add a separate date TextLayer, observing creation, layout, minute updates and cleanup on Emery.

Complete when: The watchface displays both time and date, both update correctly, and you can explain how the date layer is created, styled, attached, updated, and destroyed.

5B - Trace an existing feature through source code

Skill: Resource Literacy ? Time: 45m ? Difficulty: Medium ? Priority: Medium ? Prerequisites: 3A, 4D, 5A

Exact starter and directions: lessons/5B/.

Do: Inspect the local lessons/5B/starter font selector. Trace the Up/Down event from callback through index update to apply_font; document API signatures and state updates.

Complete when: Your notes identify the input handling, changed state, display update, and the relevant source locations and documentation.

3B - Design a font-cycling algorithm

Skill: Algorithmic Thinking ? Time: 30m ? Difficulty: Medium ? Priority: Medium ? Prerequisites: 3A, 5B

Exact starter and directions: lessons/3B/.

Do: Using the local lessons/3B/starter font selector, write original pseudocode for moving through fonts in both directions, including first/last and a one-item edge case.

Complete when: Pseudocode and a small flowchart cover both directions, normal selection, and boundary cases.

2B - Write a two-example remix brief

Skill: Idea Generation ? Time: 30m ? Difficulty: Medium ? Priority: Medium ? Prerequisites: 2A, 5B

Exact starter and directions: lessons/2B/.

Do: Choose components from the two complete local projects in lessons/2B, write a remix brief with core behavior, sources, minimal implementation and technical unknowns.

Complete when: A one-page brief explains the purpose, source features, minimal version, and one or more technical questions to investigate.

Phase III - Data, memory, state and original design

4E - Arrays, Strings, and Bounds

Skill: C Programming ? Time: 90m ? Difficulty: Medium ? Priority: High ? Prerequisites: 4D

Exact starter and directions: lessons/4E/. Reading: 4E module.

Do: Read arrays, indices, strings and bounded buffers. Extend the Emery word-selector starter to six choices and test wraparound and long labels.

Complete when: Build works; selections wrap correctly, strings remain terminated and storage is valid throughout display lifetime.

4F - Pointers, Indirection, and Memory Lifetime

Skill: C Programming ? Time: 105m ? Difficulty: High ? Priority: High ? Prerequisites: 4E

Exact starter and directions: lessons/4F/. Reading: 4F module.

Do: Read pointer parameters, indirection, ownership and lifetime. Extend the coordinate-pointer app with a safe helper, and document Pebble resource cleanup.

Complete when: Working application, pointer trace, NULL handling and ownership/lifetime explanation.

4G - Structures, Enums, and Program State

Skill: C Programming ? Time: 105m ? Difficulty: High ? Priority: Medium ? Prerequisites: 4F

Exact starter and directions: lessons/4G/. Reading: 4G module.

Do: Read structs, enums, static scope and state transitions. Extend the local mode app with PAUSED and mode-specific behavior.

Complete when: Mode transition table plus running app demonstrating valid transitions, protected operations and fallback handling.

5C - Research an unfamiliar Pebble capability

Skill: Resource Literacy ? Time: 60m ? Difficulty: High ? Priority: Medium ? Prerequisites: 4E, 5B

Exact starter and directions: lessons/5C/.

Do: Research persistence from lessons/5C/starter, trace persist_exists/read/write and write an implementation plan using the current SDK documentation.

Complete when: A technical note names the capability, links the documentation and example, lists relevant APIs and constraints, and proposes implementation steps.

6B - Diagnose a font-selection boundary bug

Skill: Debugging and Testing ? Time: 45m ? Difficulty: Medium ? Priority: Medium ? Prerequisites: 4E, 6A, 3B

Exact starter and directions: lessons/6B/.

Do: Use lessons/6B/starter with its deliberate index bug. Reproduce Up/Down behavior, inspect APP_LOG index traces, fix the faulty transition and retest every boundary.

Complete when: The bug is reproducible, the cause is explained, and tests show that cycling works correctly at both ends of the list.

2C - Develop an original Pebble app concept

Skill: Idea Generation ? Time: 60m ? Difficulty: High ? Priority: Medium ? Prerequisites: 2B, 5C

Exact starter and directions: lessons/2C/.

Do: Inspect the three complete local Emery examples in lessons/2C and develop three original ideas; compare feasibility, select one and write a minimal specification.

Complete when: Three ideas are compared and one has a concise specification for purpose, interaction, minimum version, and technical uncertainty.

3C - Design a state machine for an app

Skill: Algorithmic Thinking ? Time: 60m ? Difficulty: High ? Priority: Medium ? Prerequisites: 2C, 3B, 4G

Exact starter and directions: lessons/3C/.

Do: Use lessons/3C/starter and your 2C app concept to specify a state machine: states, inputs, transitions, visible outputs, default cases and invalid events.

Complete when: A state diagram and transition table describe all main states, triggers, resulting states, and visible responses, including important edge cases.

4H - Event-Driven Integration and Application Lifecycle

Skill: C Programming ? Time: 150m ? Difficulty: High ? Priority: Medium ? Prerequisites: 4G, 3C

Exact starter and directions: lessons/4H/. Reading: 4H module.

Do: Read callback lifecycle, event subscriptions and persistence. Extend the local app with a second mode and show event-driven updates and correct cleanup.

Complete when: Emery program demonstrates button/tick callbacks, persisted state, mode-dependent actions and safe teardown.

1C - Design a multi-mode Pebble interface

Skill: UI Design ? Time: 90m ? Difficulty: High ? Priority: Medium ? Prerequisites: 1B, 2C, 3C, 4H

Exact starter and directions: lessons/1C/.

Do: Use lessons/1C/starter (the licensed Simplicity C layout) to sketch multiple Emery screens/modes, navigation and button outcomes for your own concept.

Complete when: A set of labeled sketches covers every main state, navigation between states, button outcomes, and screen-size considerations.

Phase IV - Integration and independent verification

7C - Integrate font browsing with persistent selection

Skill: Development Tools and Workflow ? Time: 150m ? Difficulty: High ? Priority: Medium ? Prerequisites: 1C, 4H, 5B, 7B

Exact starter and directions: lessons/7C/.

Do: Combine the two complete but separate projects in lessons/7C (font selector and persistent counter). Implement saved font selection yourself, validate indices and verify restart behavior.

Complete when: The combined app builds, font navigation works, the selected font survives a restart, invalid or absent stored values are handled safely, and Git records the work.

6C - Diagnose a persistent-state integration bug

Skill: Debugging and Testing ? Time: 90m ? Difficulty: High ? Priority: Low ? Prerequisites: 4H, 6B, 7C

Exact starter and directions: lessons/6C/.

Do: Use lessons/6C/starter with its deliberate persistence fault. Select a nondefault font, restart, trace save/restore logs, correct the mismatch and retest.

Complete when: A reproducible bug report records expected and actual behavior, the cause is explained, and the fix passes fresh-start, selection, and restart tests.

Phase V - Three cumulative Emery capability projects

Fourteen new required lessons extend the curriculum with Windows, menus, button handling, graphics, sensors, resources, and phone communication. These are watchapps (unlike the original time-only watchface) to permit direct button interaction. Use the three-project progression and original chapter crosswalk.

Lesson | Topic | Duration | Depends on
8A | Windows and layers | 60m | 7A, 3A
8B | Menus and scrolling | 75m | 8A, 4E
8C | Button interactions | 60m | 8A, 4H
8D | Custom drawing | 75m | 8B, 4D
8E | Graphics paths | 75m | 8D, 4F
8F | Bitmap manipulation | 90m | 8E, 4F
8G | Accelerometer | 90m | 8C, 4H
8H | Magnetometer / compass | 90m | 8G
8I | Ambient-light limits and backlight | 45m | 8H, 5C
8J | Vibration | 45m | 8I, 8C
8K | App timers | 75m | 8C, 4H
8L | Resource files | 65m | 8F, 5C
8M | Persistent storage | 75m | 8K, 7C
8N | Phone communication | 120m | 8L, 4H, 7B

8A - Windows and layers

Skill: C Programming / SDK Lab ? Time: 60m ? Prerequisites: 7A, 3A

Exact starting project: lessons/8A/starter/ and activity instructions.

Do: Extend the time-only watchapp with a second Window and root/child layers; trace create/load/unload/destroy.

Complete when: Both screens can be displayed; illustrate layer hierarchy and clean teardown.

8B - Menus and scrolling

Skill: C Programming / SDK Lab ? Time: 75m ? Prerequisites: 8A, 4E

Exact starting project: lessons/8B/starter/ and activity instructions.

Do: Build a six-entry scrolling Field Guide menu; implement draw_row and select_click; test the first, middle, and last rows.

Complete when: Six entries scroll smoothly and selecting a row changes the visible species.

8C - Button interactions

Skill: C Programming / SDK Lab ? Time: 60m ? Prerequisites: 8A, 4H

Exact starting project: lessons/8C/starter/ and activity instructions.

Do: Add Select to open Clock Workshop details and Up/Down to adjust a bounded counter; Back returns via window stack.

Complete when: Three buttons and Back behave correctly, including minimum/maximum boundary cases.

8D - Custom drawing

Skill: C Programming / SDK Lab ? Time: 75m ? Prerequisites: 8B, 4D

Exact starting project: lessons/8D/starter/ and activity instructions.

Do: Use a custom Field Guide Layer to draw a symbol using primitives; change its output on a menu selection and mark dirty.

Complete when: A visible non-clipped drawing changes on demand and you can explain the drawing callback.

8E - Graphics paths

Skill: C Programming / SDK Lab ? Time: 75m ? Prerequisites: 8D, 4F

Exact starting project: lessons/8E/starter/ and activity instructions.

Do: Build a vertex-based species symbol using GPath; move it on selection and destroy it correctly.

Complete when: A graphics path appears in two positions and ownership/bounds are demonstrated.

8F - Bitmap manipulation

Skill: C Programming / SDK Lab ? Time: 90m ? Prerequisites: 8E, 4F

Exact starting project: lessons/8F/starter/ and activity instructions.

Do: Create a 24x24 bitmap in memory, safely alter two-color pixels by row stride, and show it in a BitmapLayer.

Complete when: Pixels visibly change with no out-of-bounds writes; bitmap and layer are freed.

8G - Accelerometer

Skill: C Programming / SDK Lab ? Time: 90m ? Prerequisites: 8C, 4H

Exact starting project: lessons/8G/starter/ and activity instructions.

Do: Subscribe to batched XYZ motion samples, rate-limit display and unsubscribe during shutdown.

Complete when: Hardware data or a documented emulator limitation with sample logs, rate, and cleanup.

8H - Magnetometer / compass

Skill: C Programming / SDK Lab ? Time: 90m ? Prerequisites: 8G

Exact starting project: lessons/8H/starter/ and activity instructions.

Do: Add magnetic heading with CompassService; show unavailable/calibrating/valid states and guard unavailable hardware.

Complete when: Heading/calibration on supported hardware, otherwise a guarded unavailable report.

8I - Ambient-light limits and backlight

Skill: C Programming / SDK Lab ? Time: 45m ? Prerequisites: 8H, 5C

Exact starting project: lessons/8I/starter/ and activity instructions.

Do: Add user-triggered backlight interaction. Explicitly distinguish the public backlight API from unavailable raw lux readings.

Complete when: Backlight interaction works and API note explains the ambient-light measurement limitation.

8J - Vibration

Skill: C Programming / SDK Lab ? Time: 45m ? Prerequisites: 8I, 8C

Exact starting project: lessons/8J/starter/ and activity instructions.

Do: Add optional vibration to a deliberate user action; never vibrate on startup or continuously for sensor data.

Complete when: Explicit trigger and off mode are demonstrated; no repeated unintended feedback.

8K - App timers

Skill: C Programming / SDK Lab ? Time: 75m ? Prerequisites: 8C, 4H

Exact starting project: lessons/8K/starter/ and activity instructions.

Do: Add a one-second elapsed counter to Clock Workshop while retaining minute tick for clock time. Reschedule/cancel safely.

Complete when: One running timer, correct elapsed output, and no callback after cleanup.

8L - Resource files

Skill: C Programming / SDK Lab ? Time: 65m ? Prerequisites: 8F, 5C

Exact starting project: lessons/8L/starter/ and activity instructions.

Do: Bundle a raw field-note resource and read it with bounded NUL-terminated storage; compare read-only resources and persistent data.

Complete when: Raw file is loaded without overflow; truncation and empty cases are considered.

8M - Persistent storage

Skill: C Programming / SDK Lab ? Time: 75m ? Prerequisites: 8K, 7C

Exact starting project: lessons/8M/starter/ and activity instructions.

Do: Save Clock Workshop’s user-defined setting and restore safely on restart; reset invalid data,

do not persist live seconds.

Complete when: Persistent, absent and invalid cases all behave as specified.

8N - Phone communication

Skill: C Programming / SDK Lab ? Time: 120m ? Prerequisites: 8L, 4H, 7B

Exact starting project: lessons/8N/starter/ and activity instructions.

Do: Implement watch-to-phone request and phone-to-watch reply starting from src/js/app.js and the messageKeys manifest; keep offline fallback.

Complete when: Two-way log with success, failure and invalid payload behavior; no required continuous connection.

Activity types

Capability Labs: Isolate one technique (e.g., TextLayers, iteration, button callbacks, arrays, persistence, or state) and test what it enables.

Remix challenges (optional extension levels): Modify one feature -> customize several -> borrow a feature -> combine two projects -> combine three -> create an original application. Remix work supplements rather than replaces the 26 required activities.

Working with local examples

Each lesson has its own folder and a README with exact instructions. Book projects have separate package.json, wscript, and src/c/main.c files. Use pebble build, pebble install --emulator emery, and pebble logs --emulator emery from the project directory.

Debugging exercises: 6A initially fails compilation with a deliberate syntax flaw; 6B compiles but has incorrect font-navigation logic; 6C compiles but restores the wrong stored value. Read diagnostics to identify the problem. Solutions are withheld.

Taskwarrior tracking

Import or update with taskwarrior/import_course.py. The importer matches existing task IDs and preserves completion status and personal annotations while refreshing dependencies, estimates and course-generated directions.

```
python3 taskwarrior/import_course.py
task project:pebble.course list
task project:pebble.course unblocked
task <ID> info
```

References

Jipping, M. (2016). *Learning C with Pebble*. GitBook. Source-derived text is clearly marked in the associated learning modules.

Official C SDK documentation: https://developer.repebble.com/docs/c/

Compatibility note: The C samples are authored/port-adapted for Emery, but the Pebble SDK was not available for automated compilation in this environment. Students must validate builds with SDK 4.33.1 and the Emery emulator.

Edition note: The previously committed SYLLABUS.pdf is the archived 26-activity edition. This Markdown document supersedes it for the full 40-activity course until the PDF is regenerated.